

Full HAETAE Jasmin NTT Verification in EasyCrypt

Codex Proof Notes

29 April 2026

Abstract

This report records the completed machine-checked correctness proof for the HAETAE NTT implementation extracted from Jasmin into EasyCrypt. The final EasyCrypt entry point is `NTTEndToEnd.ec`. It proves that the public Jasmin wrapper `Hpoly_extract.M.poly_ntt_jazz` computes the abstract full polynomial NTT `NTTFullSpec.full_ntt`, and that `Hpoly_extract.M.poly_invntt_jazz` computes the inverse abstract polynomial NTT `NTTFullSpec.full_invntt`, with the expected Montgomery-scaled output representation for the inverse. The proof chain is

public Jasmin wrapper $\longrightarrow$ Jasmin core $\longrightarrow$ imperative EasyCrypt NTT core $\longrightarrow$ closed-form abstract polynomial

The report also gives a file-by-file mathematical explanation of the remaining `.ec` files after cleanup. Files moved to `rubbish/` are scratch, debug, or probe artifacts and are not part of the proof chain described here.

Contents

1	Statement of the Verified Result	2
1.1	Mathematical domain	2
1.2	Concrete representation relation	3
1.3	Final machine-checked theorems	4
2	Overall Proof Architecture	4
3	File-by-File Mathematical Map	4
4	Detailed Proof Inventory and File Structure	6
5	Array and Extraction Support Files	12
5.1	<code>Array256.ec</code>	12
5.2	<code>BArray1024.ec</code> and <code>WArray1024.ec</code>	12
5.3	<code>extract/*.ec</code>	12
6	Finite Field and Polynomial Layer	12
6.1	<code>GFq.ec</code>	12
6.2	<code>Rq.ec</code>	13
7	Montgomery and Field Routine Correctness	13
7.1	<code>Montgomery.ec</code>	13
7.2	<code>Fq.ec</code>	14

8	Extracted Jasmin Procedure File	14
8.1	Hpoly_extract.ec	14
9	Imperative Core and Representation File	15
9.1	NTT_Fq.ec	15
9.2	Loop-shape propositions	15
9.3	Representation propositions	15
9.4	Twiddle table propositions	15
9.5	Root and Montgomery unit propositions	16
10	Closed-Form Abstract NTT Specification	16
10.1	NTTFullSpec.ec	16
10.2	Forward specification implication	16
10.3	Inverse specification implication	16
10.4	Separation from the older Rq.ntt	17
11	Loop-Level Algebraic Correctness	17
11.1	NTTFullAlgebra.ec: forward proof	17
11.1.1	Root and bit-reversal algebra	17
11.1.2	Partial sums and butterfly factorization	17
11.1.3	Pair update propositions	18
11.1.4	Stage progress propositions	18
11.1.5	Forward loop invariants and final theorem	18
11.2	NTTFullAlgebra.ec: inverse proof	19
11.2.1	Inverse address and twiddle propositions	19
11.2.2	Inverse partial sums	19
11.2.3	Inverse pair and stage progress	19
11.2.4	Inverse loop invariants and post-scaling	20
12	Jasmin-to-Core pRHL Refinement	20
12.1	RefJasminNTT.ec: word arithmetic and Montgomery facts	20
12.2	Bound predicates	21
12.3	Forward pRHL propositions	21
12.4	Inverse pRHL propositions	22
12.5	Public wrapper equivalences	22
13	End-to-End Composition	23
13.1	NTTEndToEnd.ec	23
14	Verification Evidence	23
15	Conclusion	24

1 Statement of the Verified Result

1.1 Mathematical domain

The coefficient field is

$$\mathbf{F}_q = \mathbb{Z}/q\mathbb{Z}, \quad q = 64513.$$

In EasyCrypt, this is the type `coeff` imported from `GFq.ec`, and polynomials are represented as arrays of exactly 256 coefficients:

$$\text{poly} = \mathbf{F}_q^{256}, \quad \text{type poly} = \text{coeff Array256.t.}$$

The root used by the HAETAET NTT is

$$\zeta = \text{zroot} = \text{incoeff 426.}$$

The bit-reversal permutation on eight bits is written

$$\text{br}_8(i) = \text{bsrev 8 i.}$$

Definition 1 (full forward transform). *For $p \in \mathbf{F}_q^{256}$, the full forward transform proved in `NTTFullSpec.ec` is*

$$\text{FullNTT}(p)_i = \sum_{j=0}^{255} p_j \zeta^{(2 \text{br}_8(i)+1)j}, \quad 0 \leq i < 256.$$

This is the operation `NTTFullSpec.full_ntt`.

Definition 2 (full inverse transform). *For $p \in \mathbf{F}_q^{256}$, the full inverse transform proved in `NTTFullSpec.ec` is*

$$\text{FullInvNTT}(p)_i = \sum_{j=0}^{255} 256^{-1} p_j \zeta^{-((2 \text{br}_8(j)+1)i)}, \quad 0 \leq i < 256.$$

This is the operation `NTTFullSpec.full_invntt`.

1.2 Concrete representation relation

The implementation works on `BArray1024.t`, a 1024-byte array that stores 256 little-endian 32-bit words. The main representation relation is

$$\text{NTT_Fq.poly_repr_bound rp p b.}$$

It means:

1. for every index $0 \leq i < 256$, the 32-bit word stored at `rp[i]` denotes the field element p_i ;
2. the signed integer value of that word is bounded by 2^b , written in the development as `Fq.bw32`;
3. the bound is strong enough to rule out the machine-word overflows used by the butterfly and Montgomery multiplication proofs.

The output bound for the forward NTT is 24. The output bound for the inverse NTT is 16 after the final Montgomery scaling loop.

1.3 Final machine-checked theorems

Theorem 1 (forward HAETAE Jasmin NTT correctness). *The theorem `NTTEndToEnd.haetae_jasmin_ntt_correctness` proves:*

$\{NTT_Fq.poly_repr_bound\ rp\ p\ 16\} Hpoly_extract.M.poly_ntt_jazz\ \{NTT_Fq.poly_repr_bound\ res\ (NTT_Fq.NTT.ntt\ res)\}$

Thus the public Jasmin NTT wrapper returns a byte array representing the full abstract polynomial NTT of the input polynomial.

Theorem 2 (inverse HAETAE Jasmin NTT correctness). *The theorem `NTTEndToEnd.haetae_jasmin_invntt_correctness` proves:*

$\{NTT_Fq.poly_repr_bound\ rp\ p\ 16\} Hpoly_extract.M.poly_invntt_jazz\ \{NTT_Fq.poly_repr_bound\ res\ (NTT_Fq.NTT.invntt\ res)\}$

The inverse implementation therefore returns the Montgomery-scaled representation of the abstract inverse transform. This scaling is intentional: it is the postcondition delivered by the extracted Jasmin inverse core after its final multiplication by the inverse table entry at index 255.

Remark 1. *The theorem is an implementation-correctness theorem for the HAETAE NTT routines, not for the entire HAETAE signature scheme. It is relative to the EasyCrypt model of the extracted Jasmin procedures, the field and Montgomery libraries imported by the proof, and the local representation relation.*

2 Overall Proof Architecture

The proof is organized as four mathematical bridges.

1. **Public wrapper bridge.** `RefJasminNTT.poly_ntt_jazz_ref` and `RefJasminNTT.poly_invntt_jazz_ref` show that the public wrappers call exactly the corresponding internal cores.
2. **Core pRHL bridge.** `RefJasminNTT.poly_ntt_core_ref` and `RefJasminNTT.poly_invntt_core_ref` relate the extracted Jasmin cores to the imperative EasyCrypt procedures `NTT_Fq.NTT.ntt` and `NTT_Fq.NTT.invntt`. This is where machine-word bounds, table interpretations, Montgomery multiplication, and pRHL loop synchronization are proved.
3. **Loop algebra bridge.** `NTTFullAlgebra.ntt_full_ntt` and `NTTFullAlgebra.invntt_full_invntt` prove that the imperative EasyCrypt procedures compute the closed-form polynomial transforms in `NTTFullSpec.ec`.
4. **End-to-end composition.** `NTTEndToEnd.ec` composes the previous bridges and states the final public-entry-point Hoare triples.

3 File-by-File Mathematical Map

This section explains the mathematical content of every remaining `.ec` file in the cleaned verification directory. The files in `rubbish/` are not included because they are no longer part of the proof target.

File	Mathematical role in the proof
Array256.ec	Defines the length-256 coefficient array by cloning Jasmin’s <code>PolyArray</code> . It supplies finite-array extensionality, initialization, lookup, and update principles used throughout the polynomial proofs.
BArray1024.ec	Defines 1024-byte arrays by cloning Jasmin’s byte-array theory. This is the concrete storage type for 256 32-bit coefficients.
WArray1024.ec	Defines 1024-word arrays by cloning Jasmin’s word-array theory. It is a support artifact of the extracted Jasmin memory model.
extract/Array256.ec, extract/BArray1024.ec, extract/WArray1024.ec	Generated extraction-side counterparts of the array theories. They document and support the original extraction environment; the main proof imports the top-level copies.
GFq.ec	Introduces the finite field modulus $q = 64513$, assumes its primality, and defines signed representatives, compression, and decomposition.
Rq.ec	Defines polynomials over $\mathbf{F}_q$, ring operations, an older split even/odd <code>Rq.ntt</code> , <code>Rq.invntt</code> , and base multiplication. The report explains why <code>Rq.ntt</code> is not the full 256-point HAETAET NTT.
Montgomery.ec	Provides generic signed and classical Montgomery-reduction mathematics: division facts, signed modular representatives, and REDC congruence/bound theorems.
Fastexp.ec	Imports and instantiates exponentiation support used to reason about powers of ζ , root orders, and bit-reversal exponents.
Fq.ec	Proves correctness and losslessness of the extracted Jasmin field routines <code>__montgomery_reduce</code> and <code>__fqmul</code> .
Hpoly_extract.ec	Contains the extracted Jasmin procedures and concrete twiddle tables <code>jzetas</code> and <code>jzetas_inv</code> . The NTT proof reasons about <code>_poly_ntt</code> , <code>_poly_invntt</code> , <code>poly_ntt_jazz</code> , and <code>poly_invntt_jazz</code> .
extract/Hpoly_extract.ec	Generated extraction artifact containing the same procedures and generated wrapper lemmas. The main proof uses the top-level <code>Hpoly_extract.ec</code> plus locally proved wrapper equivalences.
NTT_Fq.ec	Defines the imperative EasyCrypt NTT and inverse NTT procedures, the abstract twiddle arrays, Montgomery table interpretation, and the byte-array representation relation.
NTTFullSpec.ec	Defines the closed-form full forward and inverse NTT specifications and proves that the bit-reversed loop specification implies the closed-form formulas.
NTTFullAlgebra.ec	Contains the main loop-level mathematical proof that <code>NTT_Fq.NTT.ntt</code> and <code>NTT_Fq.NTT.invntt</code> compute <code>NTTFullSpec.full_ntt</code> and <code>NTTFullSpec.full_invntt</code> .
RefJasminNTT.ec	Contains the pRHL proof connecting extracted Jasmin cores to the imperative EasyCrypt NTT cores, including machine-word bounds, Montgomery multiplication, twiddle-table cancellation, and wrapper equivalences.

File	Mathematical role in the proof
NTTEndToEnd.ec	Composes all previous propositions into the final public-entry-point implementation-correctness theorems.

4 Detailed Proof Inventory and File Structure

This section records, file by file, what is actually proved in the EasyCrypt development and how each group of propositions enters the final claim. Files whose only content is a clone or generated procedure body still matter: they introduce semantic objects, array theories, or Jasmin procedures that later files prove correct.

File	Proofs or propositions contained in the file	Role in the final theorem
Array256.ec	No hand-written lemmas are added. The file clones Jasmin’s <code>PolyArray</code> at size 256, thereby importing the full finite-array proof theory: lookup after initialization, lookup after update, update extensionality, and equality by pointwise equality.	All polynomial specifications and loop invariants are stated over <code>Array256.t</code> . The update and extensionality facts are used in <code>NTTFullSpec.ec</code> , <code>NTTFullAlgebra.ec</code> , <code>NTT_Fq.ec</code> , and <code>RefJasminNTT.ec</code> .
BArray1024.ec	No local NTT proof is added. The file clones Jasmin’s byte-array theory at size 1024. The imported propositions give reads and writes over the concrete byte memory used to store 256 little-endian 32-bit coefficients.	It is the concrete carrier of the public Jasmin input and output arrays. The representation predicates in <code>NTT_Fq.ec</code> and the pRHL proof in <code>RefJasminNTT.ec</code> rely on its byte-level update theory.
WArray1024.ec	No local NTT proof is added. The file clones Jasmin’s word-array theory at size 1024.	It supports the extracted Jasmin memory model and keeps the extraction environment coherent with Jasmin’s word-array libraries.
extract/Array256.ec	Generated clone of the extraction-side 256-element array theory. It contains no hand-written NTT correctness proof.	Records the original extraction context. The cleaned proof uses the top-level <code>Array256.ec</code> as the imported array theory.
extract/BArray1024.ec	Generated clone of the extraction-side 1024-byte array theory.	Records the original extraction context for byte arrays. The active proof imports the top-level <code>BArray1024.ec</code> .
extract/WArray1024.ec	Generated clone of the extraction-side 1024-word array theory.	Records the original extraction context for word arrays. The active proof imports the top-level <code>WArray1024.ec</code> .
extract/Hpoly_extraction.ec	Generated Jasmin extraction file. In addition to the generated procedures, it contains generated wrapper facts <code>poly_ntt_jazz_wrapper</code> and <code>poly_invntt_jazz_wrapper</code> . These facts say that the public wrappers call the corresponding generated cores.	The active proof reproves the needed wrapper equivalences in <code>RefJasminNTT.ec</code> , but this file documents the original generated form of the same wrapper structure.

File	Proofs or propositions contained in the file	Role in the final theorem
GFq.ec	Defines $q = 64513$, assumes <code>prime_q</code> , clones <code>ZModField</code> to obtain the field $\mathbf{F}_q$, and defines signed representatives <code>as_sint</code> plus compression/decompression functions. Its main mathematical content is the field structure of coefficients.	Every algebraic proof is carried out in the cloned field $\mathbf{Z}_q$. The final correctness theorem is therefore relative to the field axiom <code>prime_q</code> .
Rq.ec	Defines <code>poly</code> , zero and one polynomials, coefficientwise addition and negation, convolution multiplication modulo $X^{256} + 1$, compression of polynomials, the root <code>zroot</code> , an older <code>Rq.ntt/Rq.invntt</code> interface, complex-style base multiplication, and <code>basemul</code> .	Supplies the polynomial type and ring notation used by the full NTT specification. The report and <code>NTTFullSpec.ec</code> distinguish its older split NTT from the full 256-point transform verified for Jasmin.
Montgomery.ec	Proves generic integer facts for powers and divisibility, signed modular representatives, and Montgomery reduction. The <code>SignedReductions</code> part proves <code>smod_bnd</code> , <code>smod_congr</code> , <code>smod_div</code> , <code>sign_comp</code> , and <code>SREDCp_corr</code> . The classical Montgomery modules prove <code>REDC'_congr</code> , <code>REDC'_bnds</code> , <code>REDC_corr</code> , and iterated <code>REDCK_corr</code> .	These propositions justify the arithmetic formula implemented by <code>__montgomery_reduce</code> and hence the field multiplication used inside every Jasmin butterfly.
Fastexp.ec	Imports and packages exponentiation facts from EasyCrypt's integer, ring, bit-encoding, and real-exponent libraries. It is used by cloning rather than by a large local list of NTT lemmas.	<code>NTT_Fq.ec</code> , <code>NTTFullSpec.ec</code> , and <code>NTTFullAlgebra.ec</code> use the resulting exponentiation support to reason about powers of <code>zroot</code> , bit-reversed exponents, and root-order identities.
Fq.ec	Instantiates <code>Montgomery.ec</code> at $q = 64513$ and proves the extracted field procedures correct. The main proof groups are word/signed-modular lemmas <code>smod_W32</code> , <code>smod_W64</code> , <code>aux32_0</code> , <code>aux32_1</code> , <code>aux32_2</code> , and <code>SAR_sem32</code> ; Montgomery-reduction theorems <code>montgomery_reduce_corr_h</code> , <code>montgomery_reduce_ll</code> , and <code>montgomery_reduce_corr</code> ; and multiplication theorems <code>fqmul_corr_h</code> , <code>fqmul_ll</code> , and <code>fqmul_corr</code> .	Turns extracted Jasmin arithmetic into field multiplication facts. These facts are used by <code>RefJasminNTT.ec</code> to prove that each machine-word butterfly step matches the corresponding $\mathbf{F}_q$ -algebraic butterfly.
Hpoly_extract.ec	Contains the extracted Jasmin module <code>M</code> , including <code>__montgomery_reduce</code> , <code>__fqmul</code> , <code>_poly_ntt</code> , <code>_poly_invntt</code> , <code>poly_ntt_jazz</code> , <code>poly_invntt_jazz</code> , and the concrete tables <code>jzetas/jzetas_inv</code> . This top-level copy has no hand-written proof lemmas.	It is the implementation being verified. <code>Fq.ec</code> verifies its arithmetic procedures; <code>RefJasminNTT.ec</code> verifies its NTT cores and public wrappers.

File	Proofs or propositions contained in the file	Role in the final theorem
NTT_Fq.ec	Defines the reference imperative procedures <code>NTT.ntt</code> and <code>NTT.invntt</code> . It proves their losslessness <code>ntt_spec_ll/invntt_spec_ll</code> ; loop-shape lemmas <code>ntt_middle_right</code> , <code>inv_middle_right</code> , <code>inv_outer_progress</code> , and <code>ntt_init_progress</code> ; representation lemmas for <code>poly_repr</code> , <code>barray256_bound</code> , and <code>poly_repr_bound</code> ; word-to-field lemmas such as <code>word_to_coeff_small</code> and <code>word_to_coeff_repr</code> ; root and Montgomery-unit lemmas <code>unit_zroot</code> , <code>inv_zroot</code> , <code>unit_R</code> , and <code>inv_R</code> ; and twiddle-table lemmas for <code>zetas</code> , <code>zetas_inv</code> , <code>jzetas</code> , and <code>jzetas_inv</code> .	Provides the verified reference core and the representation relation that connects byte arrays to polynomials. It is the middle object shared by <code>NTTFullAlgebra.ec</code> and <code>RefJasminNTT.ec</code> .
NTTFullSpec.ec	Defines <code>partial_ntt</code> , <code>full_ntt_spec</code> , <code>full_ntt</code> , <code>full_invntt</code> , and <code>full_invntt_spec</code> . It proves bit-reversal facts <code>pow2_8</code> , <code>bsrev8_range_256</code> , and <code>bsrev8_involutive_256</code> ; the specification implications <code>full_ntt_spec_imp</code> and <code>full_invntt_spec_imp</code> ; and separation lemmas <code>one_get0</code> , <code>one_get_neq0</code> , <code>rq_ntt_one_odd1</code> , <code>full_ntt_one_odd1</code> , and <code>rq_ntt_is_not_full_ntt_on_one</code> .	States the abstract target of the proof. Its implication lemmas let the loop-level algebra prove a structured invariant first and then derive the closed-form NTT formula.

File	Proofs or propositions contained in the file	Role in the final theorem
NTTFullAlgebra.ec	Contains the complete loop-level algebra proof. The forward half proves root-order and bit-reversal lemmas such as <code>exp_zroot_512</code>, <code>bsrev_stage_index</code>, <code>zetas_stage</code>, and <code>exp_zroot_stage_extra</code>; address lemmas such as <code>fwd_high_addrE</code> and <code>fwd_pair_addr_disjoint</code>; partial-sum split lemmas <code>partial_ntt_split_low/partial_ntt_split_high</code>; pair-update lemmas <code>forward_pair_refines_values</code>, <code>forward_pair_refines</code>, and <code>forward_nat_pair_refines</code>; loop invariant lemmas <code>fwd_outer_inv_init</code>, <code>fwd_middle_enter_inner</code>, <code>fwd_inner_step_array</code>, and <code>fwd_outer_exit_full</code>; and the final Hoare theorem <code>ntt_full_ntt</code>. The inverse half mirrors this with <code>zetas_inv_stage</code>, <code>partial_invntt_split_low/high</code>, <code>inverse_pair_refines_values</code>, <code>inv_inner_step_array</code>, <code>inv_inner_step_array_wp</code>, <code>inv_post_step</code>, <code>inv_post_exit_full</code>, and the final theorem <code>invntt_full_invntt</code>.	Proves that the reference imperative procedures compute the closed-form abstract transforms. This is the mathematical bridge from loop code to <code>NTTFullSpec.full_ntt</code> and <code>NTTFullSpec.full_invntt</code> .

File	Proofs or propositions contained in the file	Role in the final theorem
RefJasminNTT.ec	Contains the pRHL implementation refinement. It first proves word arithmetic facts <code>int_shr1_div2</code> , <code>int_shl1_mul2</code> , <code>word_to_coeff_add</code> , and <code>word_to_coeff_sub</code> ; butterfly range facts <code>forward_butterfly_bounds</code> and <code>inverse_butterfly_bounds</code> ; Montgomery cancellation facts <code>mont_cancel</code> , <code>mont_inv_cancel</code> , <code>jzetas_get_cancel</code> , and <code>jzetas_inv_get_cancel</code> ; <code>__fqmul</code> call facts <code>fqmul_word_to_coeff_mul</code> , <code>fqmul_word_to_coeff_mul_h</code> , and bounded variants; indexed bound predicates <code>barray256_bound_by</code> and <code>poly_repr_bound_by</code> ; forward loop-bound and butterfly lemmas including <code>forward_twiddle_value</code> , <code>forward_butterfly_store</code> , <code>forward_butterfly_store_bound_by</code> , <code>forward_butterfly_repr</code> , and <code>forward_butterfly_step</code> ; inverse counterparts including <code>inverse_twiddle_value</code> , <code>inverse_butterfly_store</code> , <code>inverse_butterfly_step_full</code> , and <code>inverse_butterfly_repr</code> ; final Montgomery scaling lemmas <code>final_mont_array_start</code> , <code>final_mont_array_exit</code> , and <code>inverse_final_product_bound_from_inv</code> ; wrapper equivalences <code>poly_ntt_jazz_ref/poly_invntt_jazz_ref</code> ; and core refinements <code>poly_ntt_core_ref/poly_invntt_core_ref</code> .	Proves that the extracted Jasmin code is behaviorally equivalent to the reference imperative NTT procedures, while maintaining the machine-word bounds needed to rule out overflow.
NTTEndToEnd.ec	Defines final postconditions <code>jasmin_ntt_full_post</code> and <code>jasmin_invntt_full_post</code> . It proves rewriting lemmas <code>full_ntt_post_from_core</code> and <code>full_invntt_post_from_core</code> ; composition lemmas <code>jasmin_ntt_full_from_core_and_alg</code> and <code>jasmin_invntt_full_from_core_and_alg</code> ; public-wrapper lemmas <code>jasmin_poly_ntt_jazz_full</code> and <code>jasmin_poly_invntt_jazz_full</code> ; and the final theorems <code>haetae_jasmin_ntt_correct</code> and <code>haetae_jasmin_invntt_correct</code> .	This is the theorem-citation file. It composes the pRHL refinement from RefJasminNTT.ec with the algebraic correctness from NTTFullAlgebra.ec, including the inverse Montgomery-scaled postcondition.

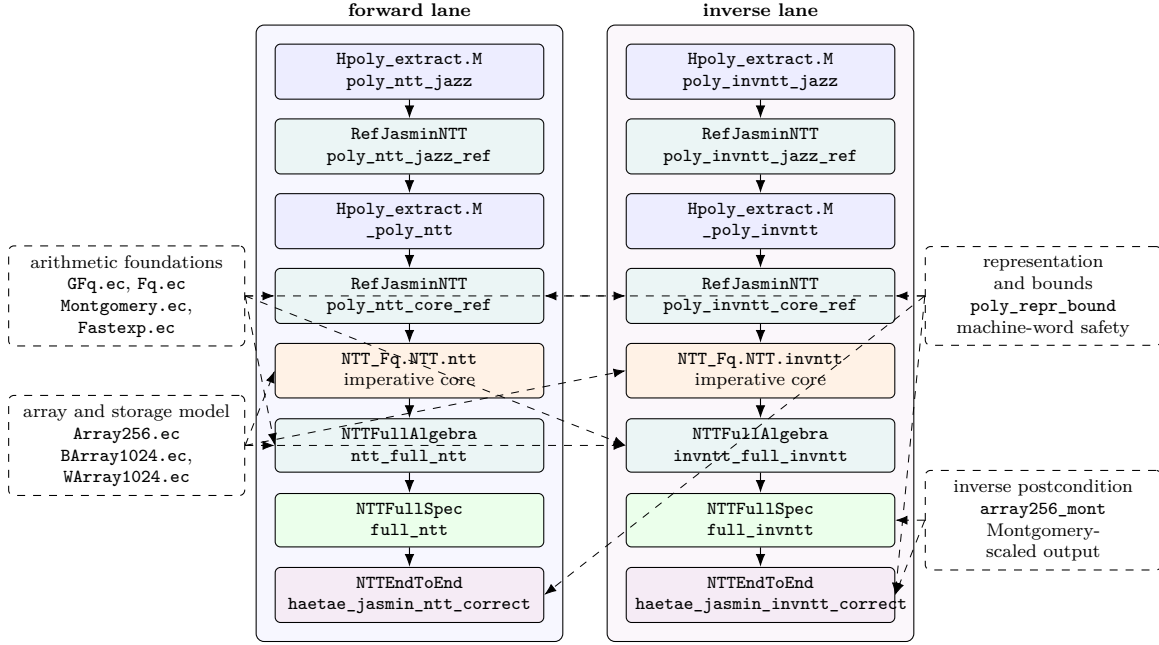


Figure 1: Structural proof script diagram for the full HAETAET NTT verification. Solid arrows are theorem-composition steps. Dashed arrows mark mathematical infrastructure used to discharge proof obligations, especially field arithmetic, Montgomery reduction, array extensionality, representation bounds, and the inverse Montgomery-scaled postcondition.

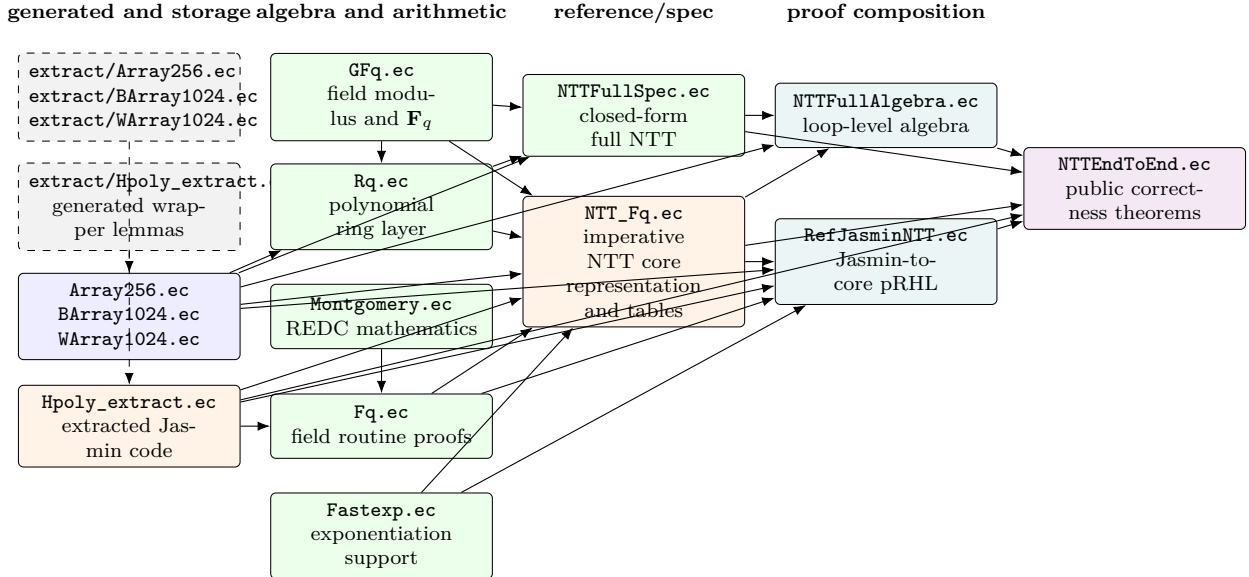


Figure 2: File-structure diagram for the verification. Solid arrows indicate imports or theorem dependencies used by later proof files. Dashed arrows show the generated extraction copies that supply the original extraction context but are not the main hand-written proof entry points.

5 Array and Extraction Support Files

5.1 Array256.ec

Array256.ec is short but essential:

```
clone export PolyArray as Array256 with op size <- 256.
```

The mathematical proposition supplied by this clone is that every `Array256.t` is an array indexed over the range $0, \dots, 255$. All polynomial equality proofs ultimately use extensionality from this clone: two arrays are equal when all 256 entries are equal. The lemmas `Array256.initiE`, `Array256.get_set_if`, `Array256.set_set_eq`, and extensional equality are used repeatedly in `NTTFullSpec.ec`, `NTTFullAlgebra.ec`, and `RefJasminNTT.ec`.

5.2 BArray1024.ec and WArray1024.ec

`BArray1024.ec` clones Jasmin’s byte-array theory at size 1024. Since $256 \cdot 4 = 1024$, this is exactly the storage required for a polynomial whose coefficients are represented as 32-bit words. Its mathematical content is the family of operations `get32` and `set32`, together with their array-update laws. These laws are the basis for proving that a concrete store at coefficient index i changes only the i -th abstract coefficient.

`WArray1024.ec` similarly clones Jasmin’s word-array theory. It is not the main polynomial representation in the final theorem, but it belongs to the extracted Jasmin environment and provides memory-model support.

5.3 extract/*.ec

The files under `extract/` are generated extraction artifacts. The three array files mirror the top-level array clones. The generated `extract/Hpoly_extract.ec` contains two wrapper lemmas:

```
poly_ntt_jazz_wrapper,    poly_invntt_jazz_wrapper.
```

They express the generated fact that the public wrappers call their internal cores and return the result. The final proof does not import these generated wrapper lemmas directly; instead, `RefJasminNTT.ec` proves equivalent wrapper equivalences over the top-level extraction file so that the proof chain remains self-contained in the cleaned directory.

6 Finite Field and Polynomial Layer

6.1 GFq.ec

`GFq.ec` fixes the modulus:

```
q = 64513,    prime_q : prime q.
```

The primality axiom gives the field structure on coefficients. The derived operations `as_sint`, `compress`, and `decompress` are general HAETAE support operations. For the NTT proof, the important mathematical proposition is that coefficients live in a prime field in which non-zero elements such as ζ , R , and 256 can have multiplicative inverses.

6.2 Rq.ec

Rq.ec defines the type

```
poly = coeff Array256.t
```

and the polynomial operations `zero`, `one`, `&+`, `&-`, and `&*`. Multiplication is negacyclic: the coefficient formula has the sign change implied by reduction modulo $X^{256} + 1$. The file also defines `zroot = incoeff 426` and `br = bsrev 8`.

The functions `Rq.ntt` and `Rq.invntt` are retained as older polynomial operators. They are not the full HAETAE NTT used in the final theorem: each splits even and odd input coefficients into separate 128-term sums. This fact is made explicit in `NTTFullSpec.ec`, where `rq_ntt_is_not_full_ntt_on_one` proves that `Rq.ntt` and `NTTFullSpec.full_ntt` differ on `Rq.one`. Thus `Rq.ec` supplies the polynomial background and a useful contrast, but the final specification is the new full 256-point transform.

7 Montgomery and Field Routine Correctness

7.1 Montgomery.ec

`Montgomery.ec` is a generic arithmetic library. Its first lemmas are integer division and divisibility facts:

```
div_pow, pow_div1, pow_div2, div_mulr, div_mull, div_mul, divM_mul.
```

These propositions justify algebraic simplification of powers of two and division by Montgomery radices.

The theory `SignedReductions` fixes a radix $R = 2^k$, an odd modulus q , and inverses required by signed Montgomery reduction. Its central operation is

$$\text{SREDC}(a) = \frac{\text{smod}(a q^{-1}, R) q + a}{R}.$$

The mathematical propositions are:

- `smod_bnd`: signed modular representatives lie in a half-open interval centered at zero;
- `smod_congr`: the signed representative is congruent to the original integer modulo the modulus;
- `smod_sq`, `smod_div`, and `sign_comp`: signed modular reduction behaves well under radix decomposition and addition;
- `SREDCp_corr`: signed Montgomery reduction is congruent to multiplication by R^{-1} modulo q , with the required range property.

The later theories `Montgomery'` and `MontgomeryLimbs` give classical REDC congruence and bound lemmas. They are parameterized by axioms for the radix, modulus, and inverse constants. These generic axioms are not the final NTT claims; they are the library assumptions under which the Montgomery correction lemmas are proved.

7.2 Fq.ec

`Fq.ec` instantiates the low-level arithmetic for $q = 64513$. The lemma `qE` records this modulus. The lemmas `smod_W32` and `smod_W64` connect EasyCrypt word signed interpretations to the integer signed modulus used in the Montgomery library.

The auxiliary lemmas

`aux32_0, aux32_1, aux32_2, SAR_sem32`

justify the bit-level arithmetic performed by the extracted Jasmin Montgomery reducer. They show that truncation, sign extension, multiplication by the constant 940508161, multiplication by 64513, and arithmetic right shifts have the intended integer meaning.

The main propositions are:

- `montgomery_reduce_corr_h`: the body of `M.__montgomery_reduce` returns a 32-bit word representing Montgomery reduction of the 64-bit input;
- `montgomery_reduce_ll`: the reducer is lossless;
- `montgomery_reduce_corr`: the public Hoare-style correctness theorem for `__montgomery_reduce`;
- `fqmul_corr_h`: the body of `M.__fqmul` returns the field product in Montgomery form;
- `fqmul_ll`: multiplication is lossless;
- `fqmul_corr`: the public correctness theorem for `__fqmul`.

These propositions are used by `RefJasminNTT.ec` to justify every concrete call to `__fqmul` in a butterfly or final inverse scaling step.

8 Extracted Jasmin Procedure File

8.1 Hpoly_extract.ec

`Hpoly_extract.ec` contains the extracted Jasmin procedures in module `M`. Its mathematical role is definitional: it gives the exact imperative programs whose correctness is proved.

The relevant procedures are:

`__montgomery_reduce, __fqmul, _poly_ntt, _poly_invntt, poly_ntt_jazz, poly_invntt_jazz.`

`_poly_ntt` is the three-loop forward butterfly implementation. It starts with $\ell = 128$, halves ℓ each outer iteration, reads one twiddle per block, computes $t = z \cdot r[j + \ell]$ by `__fqmul`, and stores

$$r[j + \ell] \leftarrow r[j] - t, \quad r[j] \leftarrow r[j] + t.$$

`_poly_invntt` is the inverse butterfly implementation. It starts with $\ell = 1$, doubles ℓ each outer iteration, stores

$$r[j] \leftarrow r[j] + r[j + \ell], \quad r[j + \ell] \leftarrow z(r[j] - r[j + \ell]),$$

and then performs the final scaling loop by the table entry at index 255.

The public wrappers `poly_ntt_jazz` and `poly_invntt_jazz` simply call the internal cores and return their result. Their equivalence to the cores is proved in `RefJasminNTT.ec`.

9 Imperative Core and Representation File

9.1 NTT_Fq.ec

NTT_Fq.ec defines the reference imperative procedures `NTT.ntt` and `NTT.invntt`. These procedures operate directly on $\mathbf{F}_q^{256}$, not on machine words. They have the same loop structure as the Jasmin cores and are the right intermediate object for pRHL synchronization.

9.2 Loop-shape propositions

The lemmas

`ntt_middle_right, inv_middle_right, inv_outer_progress, ntt_init_progress`

are arithmetic propositions about the loop counters. They show that the nested loops visit exactly the expected blocks and terminate at the expected indices. The losslessness lemmas

`ntt_spec_ll, invntt_spec_ll`

prove that the imperative reference procedures terminate without probabilistic failure.

9.3 Representation propositions

NTT_Fq.ec defines:

`word_to_coeff, barray256_to_poly, poly_repr, barray256_bound, poly_repr_bound.`

The propositions

`bw32_weaken, barray256_bound_weaken, poly_repr_bound_repr, poly_repr_bound_bound,`
`barray256_to_polyE, poly_repr_get, poly_repr_bound_get,`
`barray256_to_poly_set32E, poly_repr_set32, barray256_bound_set32, poly_repr_bound_set32`

are the algebra of the representation relation. They show how concrete 32-bit loads and stores correspond to abstract coefficient lookup and update.

9.4 Twiddle table propositions

The file defines abstract arrays `zetas` and `zetas_inv`, the Montgomery map `array256_mont`, and its inverse `array256_mont_inv`. The key table propositions include:

`zetas_invE, zetas_inv_vals, jzetas_inv_poly_vals, jzetas_inv_get, jzetas_inv_polyE,`
`zetasE, zetas_vals, jzetas_poly_vals, jzetas_get, jzetas_polyE.`

They prove that the concrete machine tables in `Hpoly_extract.ec` are the Montgomery encodings of the abstract twiddle arrays used by the reference procedures.

The special lemmas

`jzetas_inv_255_scaleE, scale255E`

identify the final inverse scaling constant as 256^{-1} in the correct Montgomery representation.

9.5 Root and Montgomery unit propositions

`unit_zroot, inv_zroot, unit_R, inv_R, exp_zroot_256.`

These lemmas state that the chosen root and Montgomery radix are units and that $\zeta^{256} = -1$. They are the algebraic facts used by the loop-level NTT proof to fold the staged butterfly equations into the closed-form transform.

10 Closed-Form Abstract NTT Specification

10.1 NTTFullSpec.ec

`NTTFullSpec.ec` is the specification layer. It introduces the correct full 256-point transforms:

`full_ntt, full_invntt.`

The preliminary lemmas

`pow2_8, bsrev8_range_256, bsrev8_involutive_256`

state that eight-bit reversal is a permutation of the 256 valid coefficient indices and is its own inverse. These propositions are repeatedly used to move between loop order and closed-form coefficient order.

10.2 Forward specification implication

`full_ntt_spec` states the loop-friendly bit-reversed output condition:

$$r_{\text{br}_8(s)} = \sum_{j=0}^{255} p_{\text{br}_8(j)} \zeta^{(2s+1) \text{br}_8(j)}.$$

The proposition `full_ntt_spec_imp` proves that this predicate implies

$$r = \text{full_ntt } p.$$

The proof applies the specification at $s = \text{br}_8(i)$, uses bit-reversal involutivity to obtain the i -th output coefficient, and reindexes the finite sum by the permutation $j \mapsto \text{br}_8(j)$.

10.3 Inverse specification implication

`full_invntt_spec` states the analogous bit-reversed inverse condition:

$$r_{\text{br}_8(i)} = \sum_{j=0}^{255} 256^{-1} p_j \zeta^{-((2 \text{br}_8(j)+1) \text{br}_8(i))}.$$

The proposition `full_invntt_spec_imp` proves that this condition implies

$$r = \text{full_invntt } p.$$

10.4 Separation from the older Rq.ntt

The lemmas

`one_get0`, `one_get_neq0`, `rq_ntt_one_odd1`, `full_ntt_one_odd1`, `rq_ntt_is_not_full_ntt_on_one`

prove that the old `Rq.ntt` is not the full NTT. At index 1, `Rq.ntt Rq.one` is zero because the old transform only sees odd input coefficients in odd output positions, while `full_ntt Rq.one` is one because the $j = 0$ term contributes 1. This proposition justifies the new full specification as the correct abstract target.

11 Loop-Level Algebraic Correctness

11.1 NTTFullAlgebra.ec: forward proof

`NTTFullAlgebra.ec` is the largest mathematical file. Its forward half proves:

$$\text{NTT_Fq.NTT.ntt}(p, \text{NTT_Fq.zetas}) = \text{NTTFullSpec.full_ntt}(p).$$

11.1.1 Root and bit-reversal algebra

The lemmas

`exp_zroot_512`, `incoeff_m1`, `exp_zroot_512_mul`, `exp_zroot_dvd512`, `exp_zroot_mod512`, `exp_zroot`

formalize the root-of-unity algebra needed by the butterfly factorization. They use $\zeta^{256} = -1$ and therefore $\zeta^{512} = 1$, allowing exponents to be reduced modulo 512.

The lemmas

`pow2_div`, `div256_pow2`, `pow2S`, `div256_pair_range`,
`bsrev_add_pow2`, `bsrev_stage_index`, `bsrev_extend_zero`, `bsrev_extend_one`,
`bsrev_double`, `bsrev_double_plus1`, `bsrev_ones`, `bsrev_zbase_index`

are the arithmetic of powers of two and bit-reversed stage indices. Their purpose is to show that the imperative loop counters identify exactly the mathematical butterfly pairs at each stage.

The table-stage lemma

`zetas_stage`

identifies the abstract twiddle read by stage k and block index ks . The pair-address lemmas

`bsrev_stage_pair_low`, `bsrev_stage_pair_high`

show that the low and high addresses of a butterfly correspond to the low and high halves of the bit-reversed decomposition.

11.1.2 Partial sums and butterfly factorization

The operation `partial_ntt` in `NTTFullSpec.ec` is used to state partially computed NTT blocks. The lemmas

`partial_ntt_split_low`, `partial_ntt_split_high`

are the main Cooley–Tukey factorization identities. They prove that a length- 2^{k+1} partial transform decomposes into two length- 2^k partial transforms, with the high part multiplied by the appropriate twiddle and sign. These lemmas are the mathematical heart of the forward proof.

11.1.3 Pair update propositions

The file defines:

`set2_add_mulr( $p, z, a, b$ )`

as the abstract array update corresponding to one forward butterfly at addresses a and b . The lemmas

`set2_add_mulr_eq1, set2_add_mulr_eq2, set2_add_mulr_neq, set2_add_mulr_assign`

record the value of the updated array at the two touched addresses and at all untouched addresses.

The lemmas

`fwd_high_addrE, fwd_old_low_addr, fwd_old_high_addr, fwd_new_low_addr, fwd_new_high_a
block_low_div, block_low_mod, block_high_div, block_high_mod, fwd_pair_addr_disjoint, fwd_a`

prove that different pairs have disjoint addresses and that address arithmetic matches the stage decomposition.

11.1.4 Stage progress propositions

The predicates

`partial_ntt_spec, full_stage_spec, fwd_old_pair_spec, fwd_new_pair_spec, fwd_stage_progress`

state what has been proved after some prefix of a stage has been executed.

The progression lemmas

`fwd_stage_progress_init, fwd_stage_progress_old, fwd_stage_progress_new,
forward_pair_refines_values, forward_pair_refines, forward_nat_pair_refines,
fwd_stage_progress_step, fwd_stage_progress_next_block, fwd_stage_progress_finish`

prove by induction over pairs and blocks that executing all butterflies of a stage transforms `full_stage_spec` at length 2^k into `full_stage_spec` at length 2^{k+1} .

11.1.5 Forward loop invariants and final theorem

The loop predicates

`fwd_outer_inv, fwd_middle_inv, fwd_inner_inv`

are the Hoare invariants for the three nested loops of `NTT_Fq.NTT.ntt`. Their maintenance is proved by:

`fwd_outer_inv_init, fwd_outer_enter_middle, fwd_middle_enter_inner,
fwd_inner_step_set2, fwd_inner_step_array, fwd_inner_exit_middle,
fwd_middle_exit_outer, fwd_outer_exit_stage, fwd_outer_exit_full.`

The lemmas `full_stage1_init` and `full_stage256_imp` connect the initial array to the first stage and the final stage to `full_ntt`.

The final checked forward theorem is:

`ntt_full_ntt : {arg = (p, NTT_Fq.zetas)} NTT_Fq.NTT.ntt {res = NTTFullSpec.full_ntt p}.`

11.2 NTTFullAlgebra.ec: inverse proof

The inverse half proves:

$$\text{NTT_Fq.NTT.invntt}(p, \text{NTT_Fq.zetas_inv}) = \text{NTTFullSpec.full_invntt}(p).$$

11.2.1 Inverse address and twiddle propositions

The inverse address functions `inv_low_addr` and `inv_high_addr` are supported by:

`inv_high_addrE`, `inv_stage_blocks_product`, `inv_pair_inputs`, `bsrev_inv_pair_low`, `bsrev_inv_pa`

These propositions show that the inverse loop order matches the inverse butterfly decomposition.

The twiddle lemmas

`zetas_inv_stage`, `inv_low_twiddle_factor`, `inv_high_twiddle_factor`

identify the inverse twiddle factors and show that they match the exponents in the closed-form inverse transform.

11.2.2 Inverse partial sums

The operation `partial_invntt` is the inverse analogue of `partial_ntt`. The lemmas

`partial_invntt_split_low`, `partial_invntt_split_high`

prove the inverse Gentleman–Sande decomposition of a length- 2^{k+1} partial inverse transform into two length- 2^k partial inverse transforms.

11.2.3 Inverse pair and stage progress

The update operation

`set2_inv_mulr`(p, z, a, b)

models one inverse butterfly. The update lemmas

`set2_inv_mulr_eq1`, `set2_inv_mulr_eq2`, `set2_inv_mulr_neq`, `set2_inv_mulr_assign`

describe the changed and unchanged entries.

The stage predicates

`inv_old_pair_spec`, `inv_new_pair_spec`, `inv_stage_progress`

are advanced by:

`inv_stage_progress_init`, `inv_stage_progress_old`, `inv_stage_progress_new`,
`inverse_pair_refines_values`, `inv_stage_progress_step`,
`inv_stage_progress_next_block`, `inv_stage_progress_finish`.

These lemmas constitute the inverse loop-level algebraic induction.

11.2.4 Inverse loop invariants and post-scaling

The loop predicates

`inv_outer_inv, inv_middle_inv, inv_inner_inv, inv_post_inv`

describe the three inverse butterfly loops and the final 256^{-1} scaling loop. Their preservation is established by:

`inv_stage1_init, inv_outer_inv_init, inv_outer_enter_middle, inv_middle_enter_inner,`
`inv_inner_step_set2, inv_inner_step_array, inv_inner_step_array_wp,`
`inv_inner_exit_middle, inv_middle_exit_outer, inv_outer_exit_stage,`
`inv_outer_exit_post, inv_post_step, inv_post_exit_full.`

The lemma `inv_inner_step_array_wp` is technically important. It bridges the array update shape produced by the invariant lemma with the contracted weakest-precondition shape produced by EasyCrypt after two writes to the same index. Mathematically it says that the same inverse butterfly update is obtained after rewriting consecutive array stores by `Array256.set_set_eq`.

The special lemma `zetas_inv_255` identifies

$$\text{NTT_Fq.zetas_inv}[255] = 256^{-1}.$$

This is used by `inv_post_exit_full` to transform the post-scaling loop invariant into `NTTFullSpec.full_invntt`.

The final checked inverse theorem is:

`invntt_full_invntt: {arg = (p, NTT_Fq.zetas_inv)} NTT_Fq.NTT.invntt {res = NTTFullSpec.full_invntt}`

12 Jasmin-to-Core pRHL Refinement

12.1 RefJasminNTT.ec: word arithmetic and Montgomery facts

`RefJasminNTT.ec` begins with integer shift facts:

`int_shr1_div2, int_shl1_mul2.`

These connect Jasmin shifts with integer division and multiplication by two.

The lemmas

`word_to_coeff_add, word_to_coeff_sub, forward_butterfly_bounds, inverse_butterfly_bounds`

state that concrete word addition and subtraction preserve the intended field values and stay within the required signed bounds.

The Montgomery cancellation lemmas

`unit_R_ref, inv_R_ref, array256_mont_get, mont_cancel, jzetas_get_cancel,`
`array256_mont_inv_get, mont_inv_cancel, jzetas_inv_get_cancel, jzetas_inv_255_scale_cancel`

show that the concrete table values are Montgomery encodings of the abstract twiddles and that the extra Montgomery factor introduced by `__fqmul` cancels exactly as required.

The lemmas

`fqmul_bw32_16, fqmul_word_to_coeff_mul, fqmul_word_to_coeff_mul_h, fqmul_word_to_coeff_mul_b`

package `Fq.fqmul_corr` for the NTT proof: they provide both the field equality and the precondition bounds needed before calling `__fqmul`.

12.2 Bound predicates

The predicates

`barray256_bound_by, poly_repr_bound_by`

allow the proof to use non-uniform bounds that vary by loop stage and by coefficient position. The lemmas

`barray256_bound_by_get, barray256_bound_by_weaken, barray256_bound_by_set32, barray256_bound`

are the algebra of these predicates.

12.3 Forward pRHL propositions

The forward stage functions

`fwd_len_ok, fwd_stage_bound, fwd_zbase, fwd_middle_bound, fwd_inner_bound`

encode the exact counter and bound facts used by the synchronized pRHL loops. Their supporting propositions include:

`fwd_stage_bound_range, fwd_stage_bound_pos, fwd_stage_bound_lt31, fwd_stage_bound_ge16,`
`fwd_stage_next, fwd_len_next_ok, fwd_zbase_next, fwd_zbase_range,`
`fwd_read_range, fwd_block_end_bound, fwd_stage_zbase_exit.`

These propositions rule out out-of-range table reads and justify stage transitions.

The local butterfly propositions

`forward_twiddle_product, forward_twiddle_product_bound_h, forward_twiddle_product_bound_p,`
`forward_twiddle_product_bound_call_h, forward_twiddle_product_bound_call_ph, forward_twiddle_`
`forward_butterfly_store, forward_butterfly_store_bound, forward_butterfly_store_bound_by`
`forward_spec_updateE, forward_butterfly_step_bound_by, forward_twiddle_product_bound_from_i`
`forward_butterfly_repr, forward_butterfly_step`

prove that one concrete Jasmin forward butterfly implements one abstract reference butterfly while preserving all bounds.

The final forward pRHL theorem is:

`poly_ntt_core_ref : Hpoly_extract.M._poly_ntt ~ NTT_Fq.NTT.ntt.`

Its precondition is

`NTT_Fq.poly_repr_bound res\{1\} res\{2\} 16 ^ zetas\{2\} = NTT_Fq.zetas,`

and its postcondition is

`NTT_Fq.poly_repr_bound res\{1\} res\{2\} 24.`

12.4 Inverse pRHL propositions

The inverse stage functions

`inv_len_ok, inv_stage_bound, inv_zbase, inv_middle_bound, inv_inner_bound`

and the lemmas

`inv_stage_bound_range, inv_stage_bound_pos, inv_stage_bound_lt31, inv_stage_bound_ge16,`
`inv_stage_next, inv_len_next_ok, inv_zbase_range, inv_read_range,`
`inv_block_end_bound, inv_stage_zbase_exit`

play the same role for the inverse loop nest.

The inverse butterfly propositions

`inverse_spec_updateE, inverse_twiddle_product_bound_call_h, inverse_twiddle_product_bound_cal`
`inverse_butterfly_store, inverse_butterfly_store_bound, inverse_butterfly_store`
`inverse_butterfly_step_bound_by, inverse_twiddle_product_bound_from_inner, inverse_twiddle_p`
`inverse_butterfly_step_full, inverse_butterfly_repr`

prove the correctness and boundedness of one inverse butterfly.

The final scaling loop is described by

`final_mont_array, final_bound.`

Its propositions are

`final_mont_array_start, final_mont_array_exit, final_mont_array`
`final_bound_start, final_bound_next_unchanged, inverse_final_product_bound_call_h, inverse_f`

They show that the loop progressively converts the reference inverse output into `array256_mont` of that output while maintaining the final 16-bit bound.

The final inverse pRHL theorem is:

`poly_invntt_core_ref : Hpoly_extract.M._poly_invntt ~ NTT_Fq.NTT.invntt.`

Its postcondition is

`NTT_Fq.poly_repr_bound res\{1\}(NTT_Fq.array256_mont res\{2\})16.`

12.5 Public wrapper equivalences

The propositions

`poly_ntt_jazz_ref, poly_invntt_jazz_ref`

prove that the public wrappers are behaviorally identical to their internal cores:

`Hpoly_extract.M.poly_ntt_jazz ~ Hpoly_extract.M._poly_ntt, Hpoly_extract.M.poly_invntt_jazz ~`

Both proofs inline the wrapper and use simulation.

13 End-to-End Composition

13.1 NTTEndToEnd.ec

NTTEndToEnd.ec introduces the two postconditions:

$$\text{jasmin_ntt_full_post } rp \ p \equiv \text{poly_repr_bound}(rp, \text{full_ntt } p, 24),$$

and

$$\text{jasmin_invntt_full_post } rp \ p \equiv \text{poly_repr_bound}(rp, \text{array256_mont}(\text{full_invntt } p), 16).$$

The small lemmas

$$\text{full_ntt_post_from_core}, \quad \text{full_invntt_post_from_core}$$

are rewriting propositions: once the core output is known to be `full_ntt p` or `full_invntt p`, the postcondition follows by unfolding the definitions.

The composition lemmas

$$\text{jasmin_ntt_full_from_core_and_alg}, \quad \text{jasmin_invntt_full_from_core_and_alg}$$

state the abstract pattern:

core refinement + algebraic correctness of the reference core $\implies$ Jasmin core correctness against the full spec.

The intermediate public-wrapper lemmas

$$\text{jasmin_poly_ntt_jazz_full}, \quad \text{jasmin_poly_invntt_jazz_full}$$

compose the wrapper equivalences with the core theorems.

The final endpoint theorems are exactly:

$$\text{haetae_jasmin_ntt_correct}, \quad \text{haetae_jasmin_invntt_correct}.$$

They are the recommended theorem names to cite when making the verification claim.

14 Verification Evidence

The cleaned proof chain was checked with:

$$\text{easycrypt compile NTTEndToEnd.ec -I .}$$

and the compile completed successfully. The direct algebraic bridge file

$$\text{easycrypt compile NTTFullAlgebra.ec -I .}$$

also compiled successfully after the inverse bridge was completed.

A scan over the proof-chain files found no active `admit`, `abort`, `axiom`, or `cheat` in the NTT proof files `NTTEndToEnd.ec`, `NTTFullAlgebra.ec`, `NTTFullSpec.ec`, and `RefJasminNTT.ec`. The background libraries `GFq.ec` and `Montgomery.ec` contain foundational axioms, such as primality of q and generic Montgomery parameters. The final correctness claim is therefore a machine-checked theorem relative to those imported mathematical foundations and the EasyCrypt/Jasmin semantic model.

15 Conclusion

The proof establishes, in EasyCrypt, that the HAETAE Jasmin NTT public wrappers implement the intended full abstract polynomial transforms. The forward wrapper computes `NTTFullSpec.full_ntt`. The inverse wrapper computes `NTTFullSpec.full_invntt` with the implementation’s expected Montgomery output scaling. The proof is not merely a core simulation: it now includes the algebraic loop-level bridge from the imperative NTT reference procedures to the closed-form polynomial NTT definitions, and then composes that bridge with the pRHL refinement of the extracted Jasmin code.